

Dashboard_Financiero

Plan de trabajo detallado

Aplicación web multiusuario para gestión de portfolios, advisor HRP, evolución histórica e informes automatizados trimestrales

Proyecto	Dashboard_Financiero
Tipo de trabajo	Aplicación web financiera con base de datos y automatización
Tecnologías principales	Python, Streamlit, SQLite, FastAPI, Pandas, SciPy, Plotly, ReportLab
Funcionalidad clave	Rebalanceo de cartera mediante Hierarchical Risk Parity (HRP)

Índice de contenidos

1. Objetivo general del proyecto
2. Alcance funcional
3. Tecnologías previstas
4. Arquitectura y estructura del proyecto
5. Diseño de la base de datos
6. Diseño de la interfaz de usuario
7. Módulo Advisor HRP
8. Evolución histórica del portfolio
9. Informes PDF y automatización trimestral
10. Datos ficticios de demostración
11. Fases de desarrollo
12. Plan temporal de ejecución
13. Prioridades, riesgos y soluciones
14. Resultado final esperado
15. Checklist final de entrega

1. Objetivo general del proyecto

El objetivo del proyecto es desarrollar una aplicación web denominada Dashboard_Financiero, orientada a la gestión y análisis de carteras de inversión de múltiples usuarios. La aplicación permitirá que cada usuario tenga asociado un portfolio propio, almacenado en una base de datos, de forma que se pueda demostrar el uso real de persistencia de datos, relaciones entre tablas y consulta dinámica de información.

Además, el sistema incorporará un módulo de recomendación o advisor financiero basado en el método Hierarchical Risk Parity (HRP), aprendido en la asignatura de Optimización de Carteras. Este módulo comparará la distribución actual de la cartera con una propuesta de pesos óptimos y generará una recomendación de rebalanceo para la fecha actual.

El proyecto también incluirá una zona de evolución histórica del portfolio, precargada con datos ficticios para simular que el usuario lleva varios años utilizando la plataforma. Finalmente, se desarrollará una API que pueda ser llamada desde herramientas externas como Zapier o Make para generar un informe trimestral y enviarlo automáticamente al correo electrónico del usuario.

Enfoque del proyecto

El objetivo no es crear una plataforma financiera real lista para producción, sino una aplicación académica completa que demuestre base de datos, automatización, análisis financiero, generación de informes y uso de un método de optimización de carteras.

2. Alcance funcional

El sistema incluirá las siguientes funcionalidades principales:

- Gestión de múltiples usuarios desde una base de datos.
- Asociación de cada usuario con un portfolio propio.
- Almacenamiento de usuarios, carteras, posiciones e histórico de evolución.
- Visualización del estado actual de cada cartera.
- Cálculo de pesos actuales de cada activo dentro del portfolio.
- Descarga o simulación de datos históricos de mercado.
- Aplicación del método HRP para obtener una asignación recomendada.
- Generación de una tabla de rebalanceo con acciones sugeridas.
- Visualización de la evolución histórica ficticia del portfolio.
- Generación de informes financieros en PDF.
- Exposición de una API para automatizar el envío trimestral de informes mediante Zapier o Make.
- Inclusión de datos ficticios precargados para facilitar la demostración del sistema.

3. Tecnologías previstas

Tecnología	Uso dentro del proyecto
Python	Lenguaje principal del proyecto.
Streamlit	Construcción de la interfaz web del dashboard.
SQLite	Base de datos local para usuarios, portfolios, posiciones e histórico.
Pandas	Tratamiento y transformación de datos financieros.
NumPy	Cálculos numéricos y generación de datos ficticios.
SciPy	Clustering jerárquico necesario para el método HRP.
yfinance	Obtención de precios históricos de activos financieros.
Plotly	Gráficos interactivos de cartera y evolución.
ReportLab	Generación de informes PDF.
FastAPI	Creación de endpoints para integraciones externas.
Pytest	Pruebas de funciones principales.

4. Arquitectura y estructura del proyecto

La aplicación se organizará en una estructura modular, separando la interfaz, la lógica financiera, el acceso a datos, la generación de informes, la API y las pruebas. Esta división permite que el proyecto sea más claro, mantenible y fácil de explicar en la memoria.

```

Dashboard_Financiero/
|
|-- app.py
|-- api.py
|-- config.py
|-- requirements.txt
|
|-- data_layer/
|   |-- db.py
|   |-- yahoo_client.py
|   |-- seed_data.py
|
|-- domain/
|   |-- portfolio_engine.py
|   |-- hrp_engine.py
|   |-- rebalance_engine.py
|   |-- evolution_engine.py
|
|-- ui/
|   |-- tab_overview.py
|   |-- tab_portfolio.py
|   |-- tab_advisor.py
|   |-- tab_evolution.py
|   |-- tab_reports.py
|
|-- reports/
|   |-- pdf_generator.py
|
|-- scripts/
|   |-- init_db.py
|
|-- tests/
|   |-- test_hrp_engine.py
|   |-- test_portfolio_engine.py
|   |-- test_rebalance_engine.py

```

Carpeta / archivo	Responsabilidad
app.py	Punto de entrada de la aplicación Streamlit.
api.py	API para generación de informes desde herramientas externas.
data_layer	Conexión con base de datos y fuentes de datos financieros.
domain	Lógica de negocio: portfolio, HRP, rebalanceo y evolución.
ui	Pestañas visuales del dashboard.
reports	Generación de informes PDF.
scripts	Inicialización y carga de datos ficticios.
tests	Pruebas unitarias del proyecto.

5. Diseño de la base de datos

La base de datos tendrá como finalidad demostrar que la aplicación trabaja con distintos usuarios y que cada uno dispone de una cartera propia. Para ello se utilizarán cuatro tablas principales: users, portfolios, positions y portfolio_history.

Tabla	Campos principales	Objetivo
users	id, name, email, created_at	Guardar los usuarios de la aplicación.
portfolios	id, user_id, name, created_at	Asociar cada usuario con su cartera.
positions	id, portfolio_id, ticker, asset_name, quantity, avg_price	Guardar los activos de cada portfolio.
portfolio_history	id, portfolio_id, date, total_value	Simular la evolución histórica de cada cartera.

5.1. Relación entre tablas

- Un usuario puede tener uno o varios portfolios.
- Cada portfolio pertenece a un único usuario.

- Cada portfolio puede contener múltiples posiciones financieras.
- Cada portfolio tendrá una serie histórica ficticia asociada.
- El dashboard consultará la base de datos en función del usuario seleccionado.

6. Diseño de la interfaz de usuario

La aplicación tendrá una interfaz principal en Streamlit con un selector lateral de usuario. Al seleccionar un usuario, todas las pestañas se actualizarán con la información correspondiente a su portfolio.

Pestaña	Contenido previsto
Resumen	Visión general del usuario, valor del portfolio, número de activos y métricas principales.
Portfolio	Tabla de posiciones, pesos actuales y gráficos de composición.
Advisor HRP	Cálculo de pesos óptimos mediante HRP y propuesta de rebalanceo.
Evolución	Gráfico histórico ficticio del valor del portfolio y métricas de rendimiento.
Informes	Generación de PDF y explicación de integración con Zapier o Make.

7. Módulo Advisor HRP

La pestaña Advisor HRP será el apartado más importante desde el punto de vista financiero. Su función será analizar la cartera actual del usuario y proponer una distribución alternativa utilizando el método Hierarchical Risk Parity.

7.1. Proceso de cálculo

1. Obtener los tickers del portfolio seleccionado.
2. Descargar precios históricos de los activos o utilizar datos simulados si la descarga falla.
3. Calcular rentabilidades diarias o semanales.
4. Calcular la matriz de correlaciones.
5. Transformar la matriz de correlaciones en una matriz de distancias.
6. Aplicar clustering jerárquico.
7. Ordenar los activos según la estructura jerárquica.
8. Aplicar asignación recursiva de pesos.
9. Obtener los pesos recomendados por HRP.
10. Comparar los pesos actuales con los pesos recomendados.

Ticker	Peso actual	Peso HRP	Diferencia	Acción recomendada
AAPL	35 %	22 %	-13 %	Reducir posición
MSFT	20 %	25 %	+5 %	Aumentar posición
NVDA	30 %	18 %	-12 %	Reducir posición
GOOGL	10 %	20 %	+10 %	Aumentar posición
AMZN	5 %	15 %	+10 %	Aumentar posición

Criterio de recomendación

Si la diferencia entre el peso HRP y el peso actual supera +3 %, se recomienda aumentar posición. Si es inferior a -3 %, se recomienda reducir posición. Si está entre -3 % y +3 %, se recomienda mantener.

Aviso académico

Las recomendaciones del advisor se plantean únicamente como simulación académica. No constituyen asesoramiento financiero profesional ni una recomendación real de inversión.

8. Evolución histórica del portfolio

La pestaña de evolución mostrará cómo habría evolucionado el portfolio del usuario durante los últimos años. Como no se dispone de histórico real de uso, se precargará una serie de datos ficticios en la base de datos. Estos datos simularán la evolución del valor total del portfolio.

- Gráfico de línea con el valor histórico del portfolio.
- Valor inicial y valor final.
- Rentabilidad acumulada.

- Rentabilidad anualizada aproximada.
- Máximo drawdown.
- Mejor periodo y peor periodo.

Esta funcionalidad permite dar más realismo al dashboard, mostrando una experiencia similar a la de una plataforma financiera que lleva varios años monitorizando la cartera del usuario.

9. Informes PDF y automatización trimestral

9.1. Informe PDF

El informe financiero incluirá los siguientes apartados:

- Datos del usuario.
- Nombre del portfolio.
- Composición actual de la cartera.
- Valor total estimado.
- Evolución histórica.
- Pesos actuales.
- Pesos recomendados por HRP.
- Tabla de rebalanceo.
- Comentario final del advisor.
- Aviso de uso académico.

9.2. API para automatización externa

Se desarrollará una API mediante FastAPI para permitir que una herramienta externa solicite la generación del informe de un usuario. El endpoint previsto será:

```
POST /api/report/{user_id}
```

Funcionamiento previsto:

1. La herramienta externa realiza una petición al endpoint.
2. La API busca el usuario en la base de datos.
3. La API obtiene su portfolio.
4. Se genera el informe PDF.
5. La API devuelve el estado de la operación, el correo del usuario, el portfolio y la ruta o enlace del PDF generado.

9.3. Flujo con Zapier o Make

Elemento	Configuración prevista
Trigger	Schedule
Frecuencia	Cada 3 meses
Acción 1	HTTP Request a la API del dashboard
Método	POST
URL	https://dashboard-financiero.com/api/report/{user_id}
Acción 2	Enviar email al correo devuelto por la API
Adjunto	Informe PDF generado automáticamente

10. Datos ficticios de demostración

Para que la aplicación pueda demostrarse sin depender de usuarios reales, se precargarán tres usuarios ficticios. Cada uno tendrá un portfolio y una evolución histórica distinta.

Usuario	Email	Portfolio	Activos
Diana Valencia	diana@example.com	Growth USA	AAPL, MSFT, NVDA, GOOGL, AMZN
Antonio Ruiz	antonio@example.com	Defensive Global	JNJ, PG, KO, PEP, V
Jose Pardo	jose@example.com	Tech Balanced	AAPL, AMD, MSFT, META, QQQ

11. Fases de desarrollo

Fase 1. Preparación del proyecto

Tareas principales:

- Crear la estructura de carpetas.
- Crear app.py, config.py y requirements.txt.
- Instalar dependencias y comprobar que Streamlit arranca correctamente.

Entregable: Estructura base del proyecto creada y aplicación inicial ejecutándose.

Fase 2. Base de datos

Tareas principales:

- Diseñar las tablas necesarias.
- Crear data_layer/db.py.
- Crear scripts/init_db.py.
- Insertar usuarios, portfolios, posiciones e histórico ficticio.

Entregable: Base de datos funcional con usuarios, portfolios, posiciones e histórico precargado.

Fase 3. Motor de portfolio

Tareas principales:

- Calcular valor actual de cada posición.
- Calcular valor total del portfolio.
- Calcular peso actual de cada activo.
- Preparar datos para tablas y gráficos.

Entregable: Motor capaz de calcular valores, pesos y composición de cartera.

Fase 4. Evolución histórica

Tareas principales:

- Generar series ficticias de evolución.
- Guardar la evolución en la base de datos.
- Calcular rentabilidad acumulada, anualizada y drawdown.

Entregable: Módulo de evolución histórica funcionando con datos ficticios.

Fase 5. Motor HRP

Tareas principales:

- Obtener precios históricos.
- Calcular rentabilidades y correlaciones.
- Aplicar clustering jerárquico.
- Calcular pesos recomendados por HRP.

Entregable: Motor HRP funcional que devuelve pesos recomendados.

Fase 6. Advisor de rebalanceo

Tareas principales:

- Comparar pesos actuales y pesos HRP.
- Calcular diferencias.
- Clasificar acciones como aumentar, reducir o mantener.

Entregable: Advisor de rebalanceo con recomendaciones interpretables.

Fase 7. Interfaz Streamlit

Tareas principales:

- Crear selector lateral de usuario.
- Crear pestañas de resumen, portfolio, advisor, evolución e informes.
- Añadir gráficos y tablas.

Entregable: Dashboard web completo con datos dinámicos por usuario.

Fase 8. Informes PDF

Tareas principales:

- Crear reports/pdf_generator.py.
- Diseñar estructura del informe.
- Permitir descarga desde la interfaz.

Entregable: Informe PDF generado automáticamente para cada usuario.

Fase 9. API

Tareas principales:

- Crear api.py.
- Crear endpoint /api/report/{user_id}.
- Conectar API, base de datos y generador de PDF.

Entregable: API funcional para solicitar informes financieros.

Fase 10. Automatización

Tareas principales:

- Definir flujo trimestral en Zapier o Make.
- Documentar llamada HTTP y envío por email.
- Incluir esquema o captura del proceso.

Entregable: Diseño documentado de automatización trimestral.

Fase 11. Pruebas

Tareas principales:

- Probar cálculo de pesos del portfolio.
- Comprobar que los pesos HRP suman 1.
- Probar generación de informes y endpoint API.

Entregable: Funciones principales validadas.

Fase 12. Documentación final

Tareas principales:

- Redactar descripción del proyecto.
- Explicar arquitectura, base de datos, HRP y automatización.
- Añadir capturas y conclusiones.

Entregable: Documentación completa y memoria preparada para entrega.

12. Plan temporal de ejecución

Teniendo en cuenta que el proyecto debe realizarse en un plazo reducido, se propone organizar el trabajo en dos jornadas intensivas. El objetivo es priorizar primero la base funcional y después completar el advisor HRP, informes, API y documentación.

Día	Bloque	Duración aproximada	Resultado esperado
Día 1	Preparación del proyecto	1 h	Aplicación base arrancando correctamente.
Día 1	Base de datos y datos ficticios	2 h	Usuarios y portfolios disponibles desde base de datos.
Día 1	Portfolio y resumen	2 h	Dashboard mostrando datos personalizados por usuario.
Día 1	Evolución histórica	1,5 h	Cada usuario tiene una evolución histórica visible.
Día 2	Motor HRP	2,5 h	Sistema capaz de generar pesos recomendados por HRP.
Día 2	Advisor de rebalanceo	1,5 h	Tabla de rebalanceo generada automáticamente.

Día 2	Informe PDF	2 h	Informe financiero descargable desde la aplicación.
Día 2	API y automatización	1,5 h	API preparada para automatización externa.
Día 2	Revisión final y documentación	2 h	Proyecto completo, documentado y listo para entregar.

13. Prioridades, riesgos y soluciones

13.1. Reparto de prioridades

Nivel de prioridad	Elementos incluidos
Alta	Base de datos multiusuario, portfolio por usuario, dashboard funcional, advisor HRP, evolución histórica, informe PDF y API.
Media	Gráficos más elaborados, métricas avanzadas, test unitarios y mejoras visuales.
Baja	Login real, despliegue en la nube, envío real de emails desde backend y seguridad avanzada.
Riesgo	Solución prevista
Fallo al descargar datos financieros	Usar datos ficticios o precios simulados como respaldo.
Complejidad del método HRP	Implementar una versión simplificada pero funcional y validar que los pesos sumen 1.
Falta de tiempo para automatización real	Desarrollar la API y documentar el flujo con Zapier o Make.
Informes PDF demasiado complejos	Crear un informe sencillo pero completo, priorizando tablas y resumen textual.

14. Resultado final esperado

Al finalizar el desarrollo, se dispondrá de una aplicación web funcional llamada Dashboard_Financiero, capaz de gestionar varios usuarios y portfolios desde una base de datos. La aplicación permitirá visualizar la composición actual de cada cartera, consultar su evolución histórica ficticia, generar recomendaciones de rebalanceo mediante HRP y crear informes financieros en PDF.

Además, el sistema incluirá una API preparada para integrarse con Zapier o Make, permitiendo automatizar el envío trimestral de informes al correo electrónico de cada usuario.

Con este proyecto se demuestra el uso conjunto de:

- Desarrollo web con Python.
- Persistencia en base de datos.
- Automatización de procesos.
- Análisis financiero.
- Optimización de carteras.
- Generación de informes.
- Integración con herramientas externas.

15. Checklist final de entrega

Elemento	Estado esperado
Aplicación Streamlit	Arranca correctamente y permite seleccionar usuario.
Base de datos SQLite	Contiene usuarios, portfolios, posiciones e histórico.
Portfolio por usuario	Cada usuario muestra una cartera distinta.
Advisor HRP	Genera pesos recomendados y tabla de rebalanceo.
Evolución histórica	Muestra datos ficticios de varios años.
Informe PDF	Se puede generar y descargar.
API	Permite solicitar informe por user_id.
Zapier/Make	Flujo trimestral explicado y documentado.
README	Incluye instalación, uso y explicación del proyecto.
Memoria	Explica arquitectura, base de datos, HRP, resultados y conclusiones.

Cierre

Este plan puede utilizarse como guía de desarrollo y como base para redactar la memoria final del proyecto. La prioridad será conseguir una aplicación funcional, visualmente clara y alineada con los requisitos: multiusuario, portfolio por usuario, advisor HRP, informes trimestrales y evolución histórica ficticia.